Java utils

Что бы это не значило

План

1. Работа с исключениями
2. Сериализация и десериализация
3. Логирование
4. Отладка
5. Мониторинг и observability

Исключения

```java
public static Candidate getCandidate(Integer id) throws Exception {
  // Ищем кандидата
  ...

  //Если не нашли
  throw new Exception("Candidate with id %d not found".formatted(id));
}


try {
  Candidate c = getCandidate(-1);
  System.out.println(c.getName());
} catch (Exception e) {
  System.out.println(e.getMessage());
}
```

Данный код не выполнится

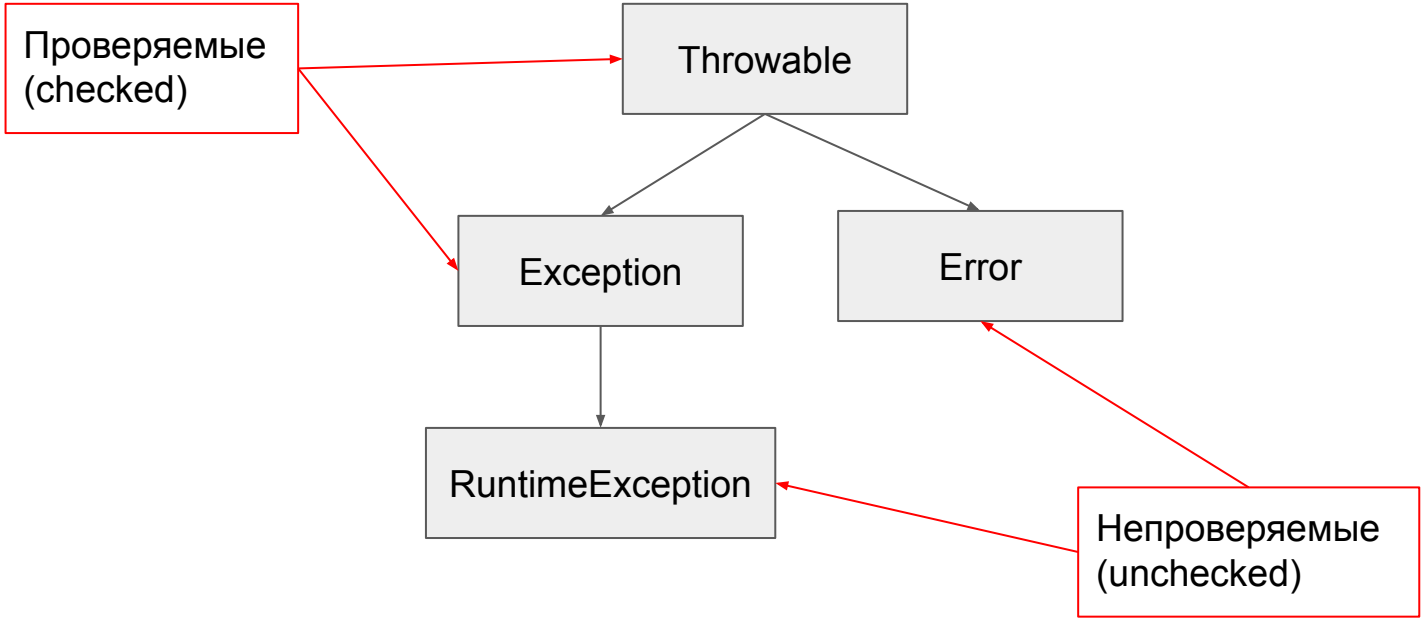
Поймали исключение

Исключения

```java
public static Candidate getCandidate(Integer id) throws Exception {
  // Ищем кандидата
  ...

  //Если не нашли
  throw new Exception("Candidate with id %d not found".formatted(id));
}


try {
  Candidate c = getCandidate(-1);
  System.out.println(c.getName());
} catch (Exception e) {
  System.out.println(e.getMessage());
}
```

```java
class CandidateNotFoundException extends Exception {}

try {
  ...
} catch (CandidateNotFoundException e) {
  // обработаем случае отсутствия кандидата
} catch (IllegalArgumentException | CompletionException e) {
  // обработаем два других исключения
} catch (Exception e) {
  // тут обработка всех других ошибок
}
```

Исключения


Проверяемые
(checked)
Throwable
Exception
Error
RuntimeException
Непроверяемые
(unchecked)

Исключения

```
public static Candidate getCandidate(Integer id) {
  // Ищем кандидата
  ...

  //Если не нашли
  throw new RuntimeException("Candidate with id %d not found".formatted(id));
}


Candidate c = getCandidate(-1);
System.out.println(c.getName());
```

Заявлять об этом не обязан

Даже если разработчик заявил, чтобы упростить жизнь,
я не обязан такой Exception ловить

Cause

```
try {
  // что-то делаем
  // но код может кинуть checked exception
} catch (SomeCheckedException e) {
  throw new RuntimeException(e);
}
```

try catch finally

```java
public class App {
  public static void main(String[] args) {
    try {
        throw new RuntimeException();
    } finally {
        System.out.println("finally");
    }
  }
}
```

try catch finally

```
public class App {
  public static void main(String[] args) {
    try {
        return;
    } finally {
        // Тут код все равно выполнится
    }
  }
}
```

try catch finally

```
try {
  // тут код выполнится
  throw new RuntimeException();
  // тут не выполнится
} catch(RuntimeException e) {
  // тут код выполнится
} finally {
  // тут тоже код выполнится
}
```

try with resources

```
Scanner scanner = null;
try {
  scanner = new Scanner(new File("test.txt"));
  // читаем файл
} catch (FileNotFoundException e) {
  // файла нет, что-нибудь другое сделаем
} finally {
  // обязательно закроем сканнер, иначе останется
  if (scanner != null) {
      scanner.close();
  }
}
```

try with resources

```java
try (Scanner scanner = new Scanner(new File("file.txt"))) {
  // читаем файл
}
```

```java
class Scanner implements AutoCloseable {
  ...

  @Override
  public void close() {
    // закрываем сканнер
  }
}
```

Использование

1. Исключение это чаще неожидаемая ситуация

Использование

1. Исключение это чаще неожидаемая ситуация

```
try {
  dto.setParam(Integer.parseInt(param));
} catch (NumberFormatException e) {
  System.out.println("Param not a number");
}
```

Использование

1. Исключение это чаще неожидаемая ситуация
2. Не обрабатывайте Error или голый Throwable

Использование

1. Исключение это чаще неожидаемая ситуация
2. Не обрабатывайте Error или голый Throwable
3. Checked используйте там, где клиент может осознанно его обработать и “исправить” ситуацию

```
public String readFile(String fileName) throw IOException {
  // читаем из файла
  // если кто-то файл удалил - кидаем IOException
}
```

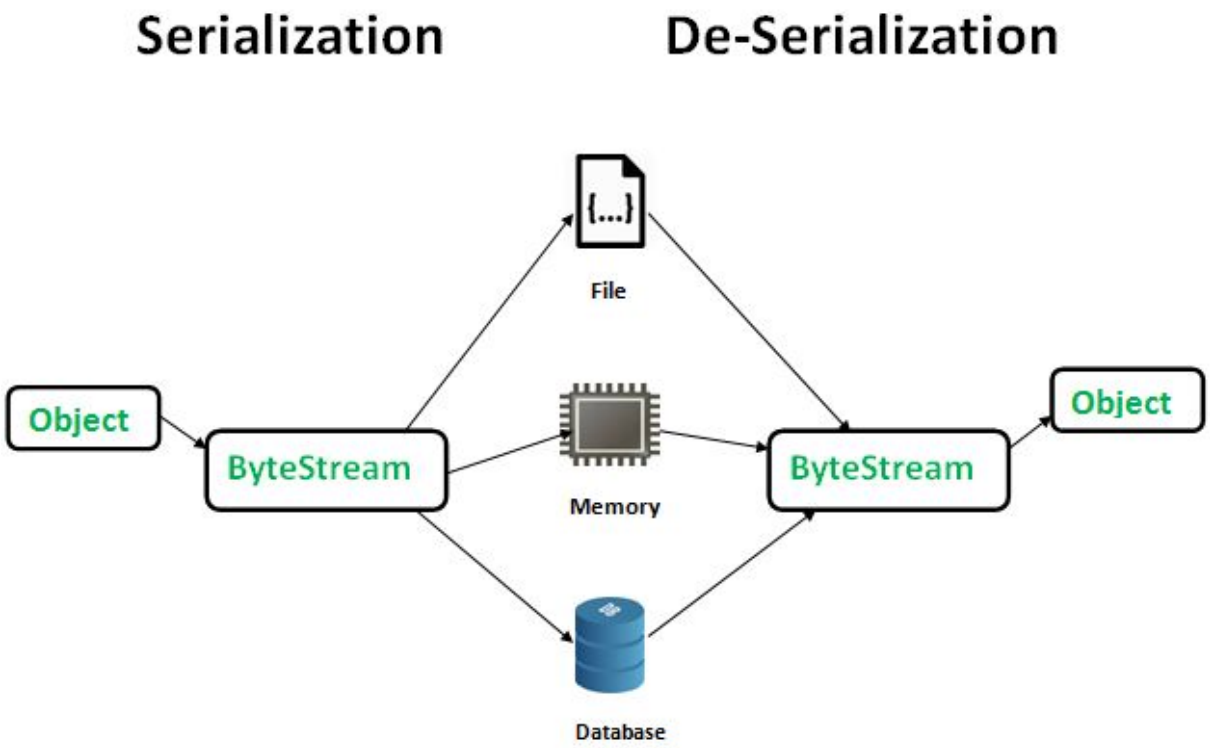
Использование

1. Исключение это чаще неожидаемая ситуация
2. Не обрабатывайте Error или голый Throwable
3. Checked используйте там, где клиент может осознанно его обработать и “исправить” ситуацию
4. Unchecked используйте там, где клиент вероятно не сможет восстановиться после ошибки

```
public void someMethod(@NotNull Person person) {
  person.getName();
  // Если вызывающий код передал null, кинется NPE
  // Он сам виноват, что не соблюдает контракт
}
```

Вопросы

Сериализация и десериализация



JSON

```
{
    "firstName": "Иван",
    "lastName": "Иванов",
    "address": {
        "streetAddress": "Московское ш., 101, кв.101",
        "city": "Ленинград",
        "postalCode": 101101
    },
    "phoneNumbers": [
        "812 123-1234",
        "916 123-4567"
    ]
}
```

Сериализация и десериализация

```
import com.fasterxml.jackson.databind.ObjectMapper;

ObjectMapper objectMapper = new ObjectMapper();
objectMapper.writeValueAsString(new Candidate(1, "Oleg"));
// {"id":1,"name":"Oleg"}

Candidate oleg = objectMapper.readValue(
    """
     {
       "id": 1,
       "name": "Oleg"
     }
    """,
    Candidate.class
);
```

Вопросы

Логирование

```
System.out.println(“Я поел”);
```

Пример лога

```
service.slog.docker192.15.log:{"ts": "2023-11-14 15:55:33.019+0300",
"msg": "Skipped (max tries exceeded). topicId=1234567, resumeHash=1234abcde4321",
"logger":"myLogger", "lvl": "ERROR"}
```

```
private static final Logger LOGGER = LoggerFactory.getLogger("myLogger");
...

LOGGER.error("Skipped (max tries exceeded). topicId={}, resumeHash={}", topicId, resumeHash);
```

Уровни

`ERROR` — серьезные ошибки. Необходимо изучить эти сообщения.

`WARN` — предупреждения. Приложение может продолжать работу, но рекомендуется обратить внимание на какую-то проблему.

`INFO` — важные сообщения. Система работает нормально.

`DEBUG` – информация для отладки.

`TRACE` – информация для отладки, когда уровня DEBUG недостаточно (редкие случаи)

Пример конфигурации (logback)

```
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <root level="debug">
    <appender-ref ref="STDOUT" />
  </root>
</configuration>
```

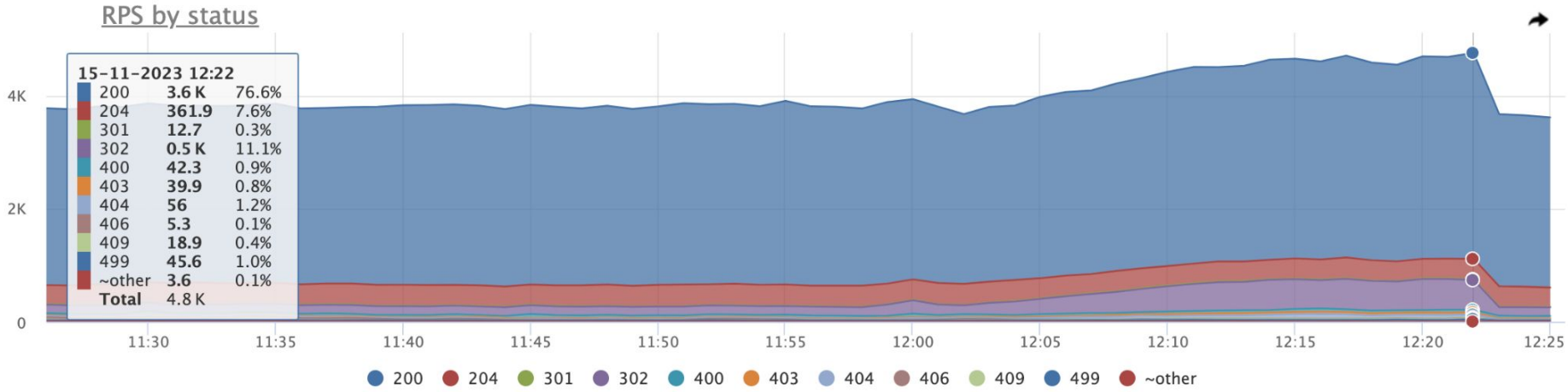
Вопросы

Дебагинг

Вопросы

Мониторинг

Какой-то график



Мониторинг - зачем и что

1. Использование процессора, памяти контейнера, чтения/записи на диск

Мониторинг - зачем и что

1. Использование процессора, памяти контейнера, чтения/записи на диск
2. Бизнес метрики - сколько резюме успешно/не успешно обработали

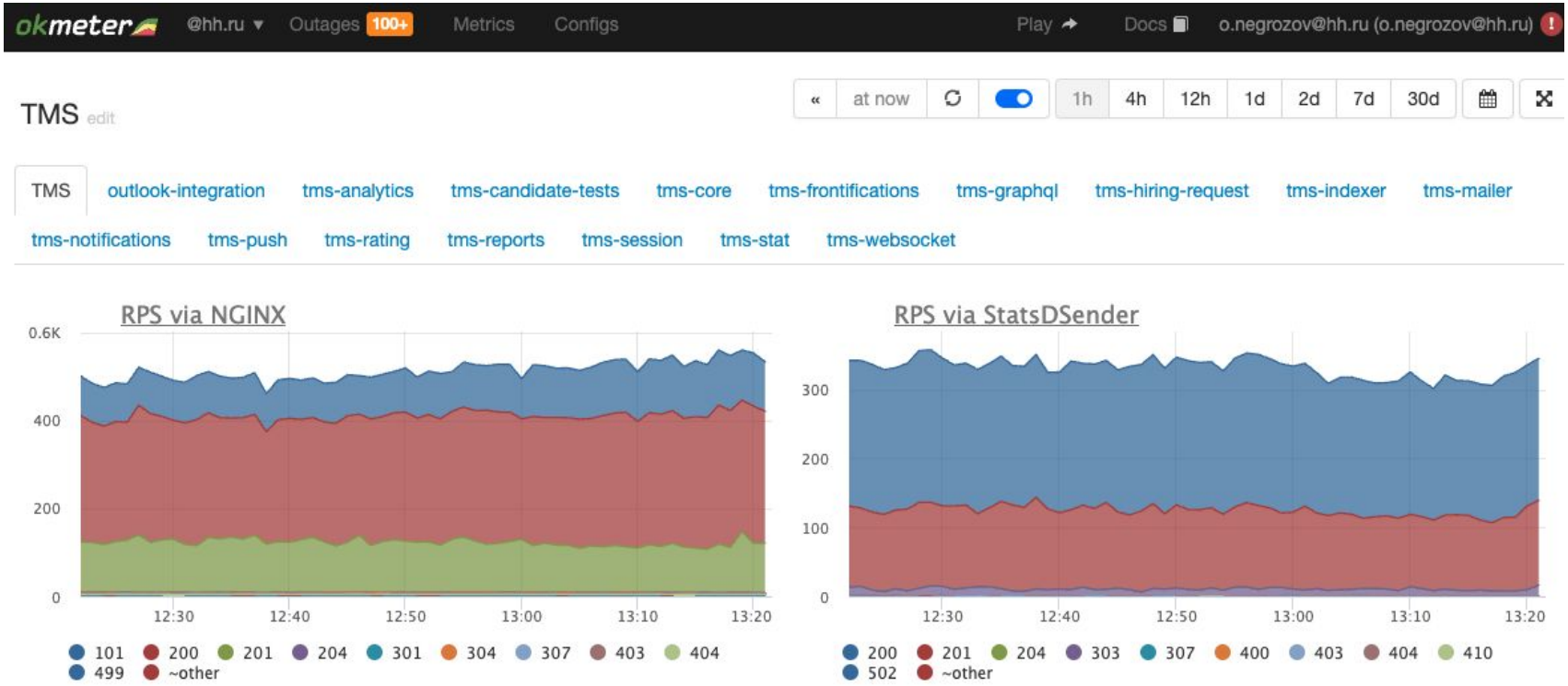
Мониторинг - зачем и что

1. Использование процессора, памяти контейнера, чтения/записи на диск
2. Бизнес метрики - сколько резюме успешно/не успешно обработали
3. JVM метрики - размеры тредпулов, stw паузы, память и т.д…

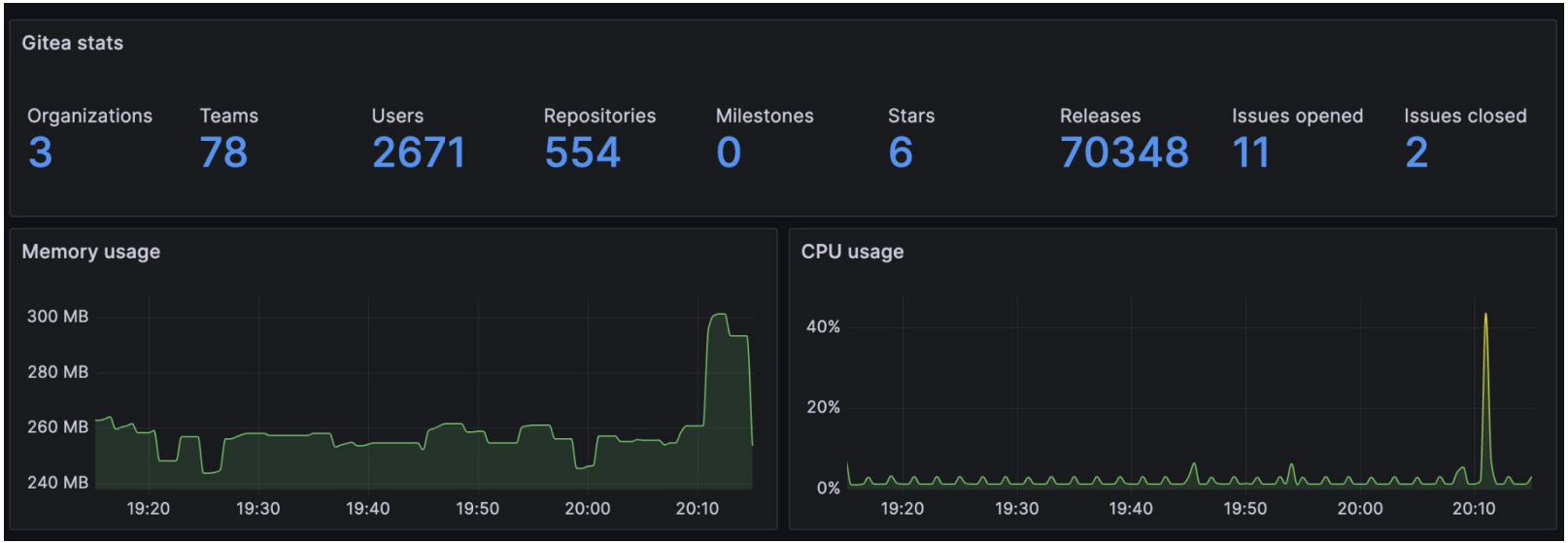
Мониторинг - зачем и что

1. Использование процессора, памяти контейнера, чтения/записи на диск
2. Бизнес метрики - сколько резюме успешно/не успешно обработали
3. JVM метрики - размеры тредпулов, stw паузы, память и т.д…
4. Что угодно

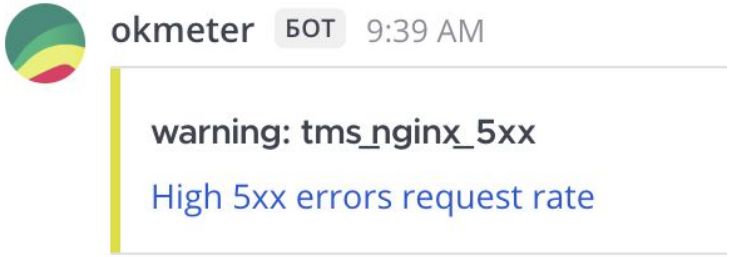
Как реагировать?



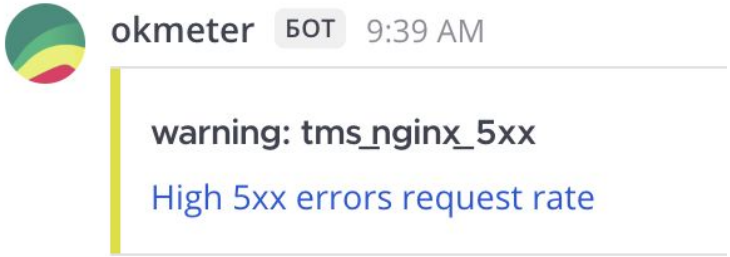
Как реагировать?



Как реагировать?



Как реагировать?




```
expression: sum(metric(name='nginx.requests.rate', status="5*", status__not="569",
file='/var/log/nginx/tms.access.slog'))

threshold: '>= 2'
severity: 'warning'
message: "High 5xx errors request rate"

team: tms
notification_config: tms_slack

notify_after: 0
resolve_after: 120
```

Способы получения метрик

1. Java agent - подсматривает за вашей jvm и снимает метрики

Способы получения метрик

1. Java agent - подсматривает за вашей jvm и снимает метрики
2. Приложение предоставляет endpoint для получения метрик

Способы получения метрик

1. Java agent - подсматривает за вашей jvm и снимает метрики
2. Приложение предоставляет endpoint для получения метрик
3. Парсинг логов


```
{"ts":"2023-11-16 03:00:51.684+0300", "cache":"-","method":"GET","ip":"172.18.20.26",
"userAgent":"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/119.0.0.0 Safari/537.36", "time":4, "rid":"17000928516799dde81d87ad65ee9640",
"uri":"/integrations/hh/import?hhid=49617202","status":200}
```

Способы получения метрик

1. Java agent - подсматривает за вашей jvm и снимает метрики
2. Приложение предоставляет endpoint для получения метрик
3. Парсинг логов
4. Метрики шлет само приложение

```
sender.sendGauge("tms.front.static.version", numberOfClients, new Tag("version", version));
```

Sentry

Resolve | Ignore | ··· | Last Seen | GRAPH: 24h 14d | EVENTS

CompletionException java.util.concurrent.CompletableFuture i...
ru.hh.jclient.common.exception.ResponseConverterException: Faile...
TMS-VJ 12min ago | 4mo old ru.hh.nab.kafka.consumer.DefaultConsumer — 1

Failed to create telegram contact for atsClientId: 58079, ...
TM... 21min ago | 4mo old *.tms.feature.integration.telegram.TelegramInt — 50

Cannot read JPEG2000 image: Java Advanced Imaging (...
TMS-XN 2hr ago | 4mo old org.apache.pdfbox.contentstream.PDFStreamI — 7

Sentry

Cannot read JPEG2000 image: Java Advanced Imaging (JAI) Image

EVENTS 88 USERS 0

Details | Activity 0 | User Feedback 0 | Attachments | Tags | All Events | Merged Issues | Replays 0

Event ID: 80dbef24 Nov 16, 8:16 AM

Recommended

Tags

environment	production	level	error	logger	org.apache.pdfbox.contentstream.PDFStreamEngine
runtime	Eclipse Adoptium 17.0.5	runtime.name	Eclipse Adoptium	server_name	docker241.prod

Message

```
Cannot read JPEG2000 image: Java Advanced Imaging (JAI) Image I/O Tools are not installed
```

Stack Trace

Most Relevant | Full Stack Trace | Newest

```
java.lang.Thread in getStackTrace at line 1610
ch.qos.logback.core.UnsynchronizedAppenderBase in doAppend at line 84
ch.qos.logback.core.spi.AppenderAttachableImpl in appendLoopOnAppenders at line 51
ch.qos.logback.classic.Logger in appendLoopOnAppenders at line 270
```

Assigned To

No one

Last 24 Hours

1

Last 30 Days

15

Last Seen

2 hours ago

First Seen

4 months ago

Вопросы

Ссылки

- Статья про исключения в java
  - https://habr.com/ru/companies/golovachcourses/articles/223821
- Стандарт json
  - https://www.json.org/json-en.html
- Репозиторий с демо по дебагу
  - https://github.com/genroelgvozo/demo-debug
- Гайд по логированию с logback
  - https://www.baeldung.com/logback

Обратная связь

Спасибо за внимание!